UNIT 2: Divide and Conquer

🔹 Divide and Conquer

```
Algorithm DAC(P)
{
    if small(P) then return S(P)
    else
    {
        Divide P into smaller subproblems P1, P2, ..., Pk
        Apply DAC to each of the subproblems Pi
        return combine(DAC(P1), DAC(P2), ..., DAC(Pk))
    }
}
```

🔹 Binary Search Algorithm

```
Algorithm b_search(a, low, high)
{
    if (low == high)
    {
        if (a[low] == element)
            print("Successful search")
        else
            print("Unsuccessful search")
    }
    else
    {
        mid = (low + high) / 2
        if (a[mid] == element)
            print("Successful search")
        else
```

```
        {
            if (a[mid] < element)
                b_search(a, mid + 1, high)
            else
                b_search(a, low, mid - 1)
        }
    }
}
```

🔹 Quick Sort Algorithm

```
Algorithm qsort(a, first, last)
{
    pivot = a[first]
    low = first
    high = last

    repeat
        low = low + 1
    until (a[low] > pivot)

    repeat
        high = high - 1
    until (a[high] < pivot)

    if (low < high) then Interchange(a, low, high)

    repeat until (low > high)

    a[first] = a[high]
    a[high] = pivot

    return high
}

Algorithm Interchange(a, low, high)
// Exchange a[low] with a[high]
```

```
{
    t = a[low]
    a[low] = a[high]
    a[high] = t
}
```

🔹 Merge Sort Algorithm

```
Algorithm MergeSort(A, low, high)
{
    if (low < high)
    {
        mid = (low + high) / 2
        MergeSort(A, low, mid)
        MergeSort(A, mid + 1, high)
        Merge(A, low, mid, high)
    }
}

Algorithm Merge(A, low, mid, high)
{
    n1 = mid - low + 1
    n2 = high - mid

    create arrays L[1..n1] and R[1..n2]

    for i = 1 to n1
        L[i] = A[low + i - 1]

    for j = 1 to n2
        R[j] = A[mid + j]

    i = 1, j = 1, k = low

    while (i <= n1 and j <= n2)
    {
        if (L[i] <= R[j])
```

```
        {
            A[k] = L[i]
            i = i + 1
        }
        else
        {
            A[k] = R[j]
            j = j + 1
        }
        k = k + 1
    }

    while (i <= n1)
    {
        A[k] = L[i]
        i = i + 1
        k = k + 1
    }

    while (j <= n2)
    {
        A[k] = R[j]
        j = j + 1
        k = k + 1
    }
}
```

🔹 Square Matrix Multiplication Algorithm

```
Algorithm Square-Matrix-Multiply(A, B)
{
    n = A.rows
    let C be a new n x n matrix

    for i = 1 to n
    {
        for j = 1 to n
```

```
        {
            C[i][j] = 0
            for k = 1 to n
            {
                C[i][j] = C[i][j] + A[i][k] * B[k][j]
            }
        }
    }

    return C
}
```

UNIT 3: Greedy Strategy

🔹 Job Sequencing with Deadline Algorithm

```
Algorithm JobSequencing(Jobs)
{
    Sort the Jobs in descending order of profit

    find the maximum deadline among all jobs, let it be maxDeadline
      create an array Slot[1..maxDeadline], initialized to -1 (free
slots)

    for each job in Jobs
    {
        for j = min(maxDeadline, job.deadline) down to 1
        {
            if (Slot[j] == -1)
            {
                Slot[j] = job.id
                break
            }
        }
    }
```

```
    Output the scheduled jobs from Slot[1..maxDeadline]
}
```

🔹 Knapsack Algorithm:

```
Algorithm Knapsack(w, p, weight, capacity, x: profit)
{
    weight = 0

    for j = 1 to n do
        x[j] = 0

    while (weight < capacity) do
    {
          # Select the object with maximum p[i]/w[i] ratio and let its
index be i

        if (weight + w[i] <= capacity) then
        {
            x[i] = 1
            weight = weight + w[i]
        }
        else
        {
            x[i] = (capacity - weight) / w[i]
            weight = capacity
        }
    }

    profit = 0

    for j = 1 to n do
        profit = profit + (x[j] * p[j])
}
```

🔹 Optimal Merge Pattern Algorithm:

```
treenode = record
{
    treenode *lchild;
    treenode *rchild;
    integer weight;
}
Algorithm Tree(n)
// list is a global list of n single node binary trees as described
above
{
    for i := 1 to n-1 do
    {
        pt := new treenode; // Get a new tree node
        (pt -> lchild) := least(list); // Merge two
        (pt -> rchild) := least(list); // trees with smallest lengths
        (pt -> weight) := ((pt -> lchild) -> weight) +
                          ((pt -> rchild) -> weight);
        insert(list, pt);
    }
    return least(list); // Tree left in list is the merge tree
}
```

🔹 Huffman Coding Algorithm:

```
Algorithm Huffman(C)
// C is set of n characters and that each character c ∈ C is an object
with defined frequency f(c)
// Q - min priority queue with the characters in C

n <= |C|
Q <= C
for i = 1 to n-1
```

```
   do allocate a new node z
   left[z] <= x = Extract-min(Q)
   right[z] <= y = Extract-min(Q)
   f(z) <= f(x) + f(y)
   Insert(Q, z)
return Extract-min(Q) → Return root of tree
```

🔹 Kruskal's Algorithm

```
Algorithm Kruskal(G)
{
    A = ∅                           // A will store the edges of the MST
    sort all edges of G in non-decreasing order of weight
    for each vertex v in G
         make-set(v)                // Create a disjoint set for each
vertex

    for each edge (u, v) in sorted edge list
    {
        if find-set(u) ≠ find-set(v)
        {
            A = A ∪ {(u, v)}     // Add edge to MST
            union(u, v)          // Combine the sets
        }
    }

     return A                        // A contains the edges of the
Minimum Spanning Tree
}
```

🔹 Prim's Algorithm

```
Algorithm Prim(G, start)
{
    for each vertex u in G
    {
        key[u] = ∞
```

```
        parent[u] = NIL
        visited[u] = false
    }

    key[start] = 0

    for i = 1 to |V| do
    {
         u = extract-min(key, visited)  // Select the unvisited vertex
with min key
        visited[u] = true

        for each adjacent vertex v of u
        {
            if visited[v] == false and weight(u, v) < key[v]
            {
                key[v] = weight(u, v)
                parent[v] = u
            }
        }
    }

    return parent[]  // parent[] represents the MST
}
```

UNIT 4: Dynamic Programming

🔹 Backward Cost Algorithm

```
Algorithm B_cost(cost, n, d, bcost: r)
{
    bcost[d] = 0

    for j = d - 1 to 0 do
    {
          // Find a vertex t such that cost(t, j) ≠ ∞ and bcost[t] +
cost(t, j) is minimum
```

```
        bcost[j] = bcost[t] + cost(t, j)
        r[j] = t
    }

    // Find minimum cost path
    p[0] = 1
    p[k] = n
    for j = k - 1 to 2 do
        p[j] = r[p[j + 1]]
}
```

🔹 Forward Cost Algorithm

```
Algorithm F_cost(cost, n, d, fcost: p)
{
    fcost[0] = 0

    for j = 1 to d do
    {
          // Find a vertex i such that cost(i, j) ≠ ∞ and fcost[i] +
cost(i, j) is minimum
        fcost[j] = fcost[i] + cost(i, j)
        p[j] = i
    }

    // Find minimum cost path
    p[0] = 1
    p[k] = n
    for j = 2 to k - 1 do
        p[j] = p[p[j - 1]]
}
```

🔹 Longest Common Sub - Sequence Algorithm:

🔸 **Part 1: Build LCS Table**

```
Algorithm LCS_Length(a, b)
{
    m = length(a)
    n = length(b)

    for i = 0 to m do
        c[i][0] = 0

    for j = 0 to n do
        c[0][j] = 0

    for i = 1 to m do
        for j = 1 to n do
        {
            if (a[i] == b[j]) then
                c[i][j] = c[i - 1][j - 1] + 1
            else
                c[i][j] = max(c[i - 1][j], c[i][j - 1])
        }
}
```

🔸 **Part 2: Reconstruct the LCS**

```
Algorithm Print_LCS(c, a, i, j)
{
    if (i == 0 or j == 0)
        return

    if (a[i] == b[j]) then
    {
        Print_LCS(c, a, i - 1, j - 1)
        output a[i]
    }
    else if (c[i - 1][j] > c[i][j - 1]) then
        Print_LCS(c, a, i - 1, j)
    else
        Print_LCS(c, a, i, j - 1)
}
```

🔹 All-Pairs Shortest Path Algorithm (Floydd Warshall)

```
Algorithm APSP(cost, n, A)
{
    for i = 1 to n do
        for j = 1 to n do
            A[i][j] = cost[i][j]
    for k = 1 to n do        // intermediate vertex
        for i = 1 to n do
            for j = 1 to n do
                A[i][j] = min(A[i][j], A[i][k] + A[k][j])
}
```

🔹 Bellman-Ford Algorithm

```
Algorithm BellmanFord(V, cost, dist)
{
    // Single source / all destination
    // Works with negative edge costs

    for i = 1 to n do
        dist[i] = cost[v][i]      // initialize dist from source v

    for k = 1 to n - 1 do         // n - 1 iterations
    {
        for each node u ≠ v and u has at least one incoming edge do
        {
            for each edge (x, u) in the graph do
            {
                if dist[u] > dist[x] + cost[x][u] then
                    dist[u] = dist[x] + cost[x][u]
            }
        }
    }
}
```

UNIT 5: Backtracking

🔹 N-Queens Algorithm

```
Algorithm NQueens(k, n)
// Using backtracking this procedure prints
// possible placements of n queens on a
// chessboard so that they are non-attacking
{
    for i := 1 to n do
    {
        if place(k, i) then
        {
           x[k] := i;
           if (k = n) then write(x[1:n]);
           else NQueens(k + 1, n);
       }
   }
}
```

🔹 Graph Coloring Algorithm

```
Algorithm mColoring(k)
// k is the index of the next vertex to color
{
    repeat
    {
        NextValue(k); // Generate all legal assignments to x[k]
        if (x[k] = 0) then return; // No new color possible
          if (k = n) then // At most m colors have been used to color
the vertices
            write(x[1:n]);
        else mColoring(k + 1);
```

```
    } until (false);
}
```

🔹 Hamiltonian Cycle Algorithm

```
Algorithm H-cycle(k)
{
    repeat
    {
        NextValue(k);
        if (x[k] = 0) then
            return;
        if (k = n) then
            if (x[n] is adjacent to x[1]) then
                write(x[1:n]);
        else Hamilton(k + 1);
    } until (false);
}
```

🔹 Non - deterministic Search Algorithm

```
Algorithm NDA_search(A, n)
{
    j := choice(1...n);
    if A[j] = x then
    {
        write(j);
        success();
    }
    write(0);
    failure();
}
```

🔹 Non - deterministic Sort Algorithm

```
Algorithm NDA_sort(A, n)
{
    for i := 1 to n do
        B[i] := 0;

    for i := 1 to n do
    {
        j := choice(1...n);
        if B[j] ≠ 0 then
            failure();
        B[j] := A[i];
    }

    for i := 1 to n - 1 do
        if B[i] > B[i + 1] then
            failure();

    write(B[1...n]);
    success();
}
```

🔹 Clique Algorithm

```
Algorithm Clique(G, n, k, S)
{
    // Size of clique = k
    // S represents a set in which vertices are sorted.

    S = ∅  // S is initially empty set

    for i = 1 to k do
    {
        t = choice(1...n)

        if (t ∈ S) then
            failure();
        else
```

```
            S = S ∪ {t}  // t distinct vertices are selected
    }

    // At this point S contains k distinct vertex indices
    for all pairs (i, j) such that i ∈ S, j ∈ S and i ≠ j do
    {
        if ((i, j) ∉ E) then
            failure();
    }

    success();
}
```

🔹 Graph Partitioned into Triangle Algorithm

```
Algorithm GPT(G, n, k, S)
{
    S ≠ ∅  // Initially set is empty

    for i = 1 to k do
    {
        t = choice(1...n)
        if (t ∈ S) then
            failure()
        else
            S = S ∪ {t}  // Add t to S
    }

    for each pair (i, j) ∈ S and i ≠ j do
    {
        if ((i, j) ∉ E) then
            failure();
    }

    success();
}
```

🔹 Independent set Problem Algorithm

```
Algorithm ISP(G, n, t, S₁, S₂)
{
    // t → size of S₁
    S₁ = ∅, S₂ = ∅

    for i = 1 to t do
    {
        j = select(1...n)

        if (j ∈ S) then
            failure();
        else
        {
            S₁ = S₁ ∪ {j}
            S₂ = G - S₁  // Remaining vertices
        }
    }

    // Connectivity check
    for i = 1 to t do  (i ∈ S₁)
        for j = 1 to (G - t) do  (j ∈ S₂)
        {
            if ((i, j) ∉ E) then
                failure();
        }

    success();
}
```

🔹 Non-deterministic Knapsack Problem Algorithm

```
Algorithm ND-Knapsack(p, w, n, m, r, x)
```

```
{
    // p: profit array
    // w: weight array
    // n: number of items
    // m: capacity of knapsack
    // r: expected profit from knapsack
     // x: array to indicate whether corresponding item is carried or
not

    w = 0
    P = 0

    for i = 1 to n do
    {
        x[i] = choice(0, 1)
        w = w + w[i] × x[i]
        P = P + p[i] × x[i]
    }

    if ((w > m) || (P < r)) then
        failure();
    else
        success();
}
```